TECHNICAL ANALYSIS REPORT

YLD Arecanut Disease Classification System

A comprehensive analysis of the AdikeScan deep learning application for arecanut palm disease detection, explainability, and agronomic decision support.

REPOSITORY

github.com/Kr1szz/YLD

GENERATED BY

Z.ai Analysis Engine

DATE

August 2026

Table of Contents

1. Executive Summary **3**

2. System Architecture **3**

2.1 Overall Architecture 3

2.2 Backend Architecture 4

2.3 Frontend Architecture 4

3. Deep Learning Model Analysis **5**

3.1 Model Architecture 5

3.2 Training Configuration 5

3.3 Training Results and Performance 6

4. Disease Classification System **7**

4.1 Nine-Class Taxonomy 7

4.2 Disease Knowledge Base 8

5. Grad-CAM Explainability **8**

5.1 Explainability Mechanism 8

5.2 Severity Estimation 9

6. API Design and Integration **9**

6.1 Endpoint Specification 9

6.2 Prediction Response Structure 10

7. Code Quality and Technical Assessment **10**

7.1 Strengths 10

7.2 Areas for Improvement 11

7.3 Technology Stack Summary 11

8. Recommendations **12**

8.1 Short-Term Improvements 12

8.2 Long-Term Development 12

1. Executive Summary

YLD (Yellow Leaf Disease) is a full-stack, AI-powered web application designed for the classification and explainability of diseases affecting arecanut palms (*Areca catechu*). The system, also known as **AdikeScan**, combines a fine-tuned MobileNetV3-Small deep learning model with a React-based frontend to deliver real-time disease diagnosis and Grad-CAM neural explainability. The repository is maintained by developer Kr1szz on GitHub and represents a practical application of computer vision techniques in agricultural pathology, specifically targeting the arecanut farming community in South and Southeast Asia where this crop holds significant economic and cultural importance.

The system classifies uploaded plant images into nine distinct categories: four healthy tissue types (Healthy Foot, Healthy Leaf, Healthy Nut, Healthy Trunk) and five disease or pest categories (Bud Borer, Mahali Koleroga, Stem Cracking, Stem Bleeding, and Yellow Leaf Disease). The trained model achieves an impressive 99.52% validation accuracy after just 15 epochs of fine-tuning, demonstrating the effectiveness of transfer learning from ImageNet-pretrained weights. The model checkpoint is compact at approximately 6 MB, making it suitable for deployment in resource-constrained environments such as mobile devices or edge computing nodes in rural farming areas.

A distinguishing feature of this project is its integration of authentic Gradient-weighted Class Activation Mapping (Grad-CAM) for visual explainability. Rather than simply returning a classification label, the system generates a saliency heatmap overlaid on the original image, highlighting the specific tissue regions that influenced the model prediction. This is complemented by a quantitative severity estimation mechanism that calculates the percentage of affected tissue area and classifies it into Mild, Moderate, Severe, or Critical categories. This combination of classification accuracy and interpretability makes the system particularly valuable for agricultural extension workers and plant pathologists who need to understand and trust AI-driven diagnostic recommendations.

2. System Architecture

2.1 Overall Architecture

AdikeScan follows a clean client-server architecture with a clear separation between the deep learning backend and the user-facing frontend. The backend is implemented in Python using the FastAPI framework, serving as both the model inference engine and the REST API provider. The frontend is a single-page application (SPA) built with React 19 and bundled with Vite 6, communicating with the backend exclusively through HTTP API calls. This decoupled design allows independent development, testing, and deployment of each component, and facilitates potential future scalability improvements such as containerization with Docker or orchestration with Kubernetes.

The data flow is straightforward: the user uploads an image through the React frontend, which sends it as a multipart form POST request to the backend `/predict` endpoint. The backend preprocesses the image, runs it through the PyTorch model, computes Grad-CAM if requested, and returns a structured JSON response

containing the predicted class, confidence score, probability distribution across all nine classes, detailed disease information from the embedded knowledge base, and base64-encoded Grad-CAM visualizations. The frontend then renders this data through a series of specialized React components that present the results in an intuitive, visually appealing manner.

2.2 Backend Architecture

The backend is organized around six Python modules within the `YLD/backend/app/` directory, each responsible for a distinct concern. The **main.py** module defines the FastAPI application with three endpoints: a health check endpoint that reports model readiness and system status, a disease catalog endpoint that returns the complete agronomic knowledge base, and the primary prediction endpoint that accepts image uploads and returns classification results with optional Grad-CAM computation. The application includes a demo mode fallback mechanism that gracefully handles scenarios where the trained model checkpoint is not available, returning a uniform probability distribution instead of crashing.

The **model.py** module encapsulates the MobileNetV3-Small architecture configuration and model loading logic. It constructs the model by loading ImageNet-pretrained weights from TorchVision and replacing the final classification layer with a custom linear head outputting nine classes. The module also defines the image transformation pipelines for both training (with augmentation) and validation (without augmentation), using standard ImageNet normalization statistics. The **train.py** script implements the complete training loop with AdamW optimizer, cosine annealing learning rate scheduling, cross-entropy loss with label smoothing, and best-model checkpointing based on validation accuracy.

The **gradcam.py** module provides the explainability layer, leveraging the pytorch-grad-cam library to compute gradient-weighted activation maps through the final convolutional layer of MobileNetV3-Small. It generates both an overlay visualization (heatmap composited onto the original image) and a raw heatmap, along with a severity score calculated as the fraction of activation pixels exceeding a configurable threshold (default 0.35). The **disease_info.py** module serves as the agronomic knowledge base, defining the nine class names, scientific nomenclature, detailed symptom descriptions, treatment protocols, and preventive measures for each disease category. Finally, **split_dataset.py** provides a utility for splitting a raw dataset into 80/20 train/validation partitions with reproducible random seeding.

2.3 Frontend Architecture

The frontend is a React 19 single-page application developed with Vite 6 as the build tool. The component hierarchy is led by **App.jsx**, which manages the global application state including the selected image, prediction results, system health status, and the disease catalog modal. It orchestrates API communication with the backend and delegates rendering to four specialized child components. **LeafUpload.jsx** provides a drag-and-drop image upload widget with file browser and camera capture support, featuring a visually rich dark-themed design with preview capabilities and file metadata display.

PredictionCard.jsx renders the classification results with a tabbed interface that presents three views: Symptoms and Diagnosis (disease description and diagnostic indicators), Treatment and Control (recommended direct measures and preventive cultural practices), and All Class Probabilities (a visual bar chart showing the confidence distribution across all nine classes). **GradCamViewer.jsx** displays the

Grad-CAM visualizations with toggle controls for overlay, heatmap, and original image views, alongside a severity metrics panel showing the estimated tissue severity ratio as a percentage with a color-coded progress bar. **DiseaseCatalog.jsx** implements a modal-based reference guide that presents the complete nine-disease knowledge base in a browsable format. The entire frontend uses a custom dark emerald CSS theme with glassmorphism-style panels and lucide-react icons.

3. Deep Learning Model Analysis

3.1 Model Architecture

The core of the classification system is a MobileNetV3-Small architecture, a lightweight convolutional neural network originally developed by Google for mobile and edge deployment scenarios. The model was selected for its favorable balance between computational efficiency and classification accuracy, making it particularly suitable for agricultural applications where deployment hardware may be limited. The base architecture, pretrained on the ImageNet dataset comprising over one million images across one thousand classes, is adapted for the arecanut disease classification task by replacing the final classification head with a custom linear layer that outputs logits for the nine target classes.

MobileNetV3-Small employs depthwise separable convolutions, squeeze-and-excitation blocks, and an inverted residual structure to achieve high accuracy with minimal computational cost. The final feature extraction layer (`features[12]`) produces a spatial feature map that serves as the target layer for Grad-CAM computation. The model accepts 224 x 224 pixel RGB inputs and processes them through ImageNet-standard normalization (mean: [0.485, 0.456, 0.406], std: [0.229, 0.224, 0.225]). The resulting model checkpoint is approximately 6 MB in size, which is remarkably compact for a nine-class classifier and enables practical deployment scenarios including web servers, mobile applications, and embedded systems.

3.2 Training Configuration

The training pipeline is implemented in the **train.py** module and follows established best practices for transfer learning on small-to-medium agricultural image datasets. The configuration choices reflect careful consideration of the trade-offs between overfitting prevention and convergence speed, and the results demonstrate that these choices were well-calibrated for the dataset characteristics. The following table summarizes the key training hyperparameters and their chosen values:

Parameter	Value	Rationale
Base Architecture	MobileNetV3-Small	Lightweight, mobile-deployable, ImageNet pretrained
Pretrained Weights	ImageNet (DEFAULT)	Transfer learning from 1000-class general features
Number of Classes	9	4 healthy + 5 disease categories

Parameter	Value	Rationale
Input Resolution	224 x 224 px	Standard for MobileNetV3
Epochs	15	Sufficient convergence observed by epoch 2
Batch Size	32	Balanced for GPU memory and gradient stability
Learning Rate	3e-4	Conservative for fine-tuning
Optimizer	AdamW	Adam with decoupled weight decay
Weight Decay	1e-2	Regularization against overfitting
LR Scheduler	Cosine Annealing	Smooth decay, avoids sudden LR drops
Loss Function	CrossEntropyLoss	Standard for multi-class classification
Label Smoothing	0.05	Mild regularization, prevents overconfident predictions
Train/Val Split	80 / 20	Standard split ratio, seed 42
Augmentation	H-flip, V-flip, Rotate, Color Jitter	Reduces overfitting on small dataset

Table 1: Training hyperparameters and their design rationale.

3.3 Training Results and Performance

The training history reveals a textbook example of successful transfer learning with rapid convergence. The model achieved 91.35% validation accuracy after the very first epoch, leveraging the rich feature representations learned from ImageNet. By epoch 2, the validation accuracy jumped to 97.60%, and by epoch 3, it reached the plateau of 99.52% where it remained stable through all subsequent epochs. This rapid convergence pattern is characteristic of transfer learning scenarios where the pretrained features are already highly discriminative and only require minor adaptation to the target domain.

The training accuracy followed a similar trajectory, starting at 69.39% in epoch 1 (reflecting the regularization effect of data augmentation during training), rising to 96.26% in epoch 2, and reaching 100% by epoch 7. The gap between training and validation accuracy remained minimal throughout training, with the validation loss decreasing monotonically from 0.6594 to 0.2942, indicating no significant overfitting. The training loss decreased from 1.2885 to 0.2924 over 15 epochs, showing consistent optimization progress. The final best model checkpoint was saved at epoch 2 with a validation accuracy of 99.52%, demonstrating that the early stopping mechanism (implicit via best-model tracking) would have been effective had it been explicitly implemented.

Epoch	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)
1	1.2885	69.39	0.6594	91.35
2	0.4351	96.26	0.4105	97.60

Epoch	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)
3	0.3431	99.73	0.3537	99.52
5	0.3100	99.73	0.3164	99.52
7	0.2997	100.00	0.3001	99.52
10	0.3012	99.60	0.2942	99.52
15	0.2924	100.00	0.2966	99.52

Table 2: Selected training metrics showing convergence trajectory (selected epochs).

The 99.52% validation accuracy is a strong result that positions the model well for practical deployment. However, it is important to note several considerations regarding this metric. First, the validation set represents only 20% of the total dataset, and performance on truly out-of-distribution images (different cameras, lighting conditions, geographic regions, or growth stages) may differ. Second, the dataset likely contains class imbalance between the relatively common healthy categories and the rarer disease categories, which could inflate overall accuracy. A more comprehensive evaluation including per-class precision, recall, F1-score, and a confusion matrix would provide deeper insight into the model practical performance.

4. Disease Classification System

4.1 Nine-Class Taxonomy

The classification system employs a carefully designed nine-class taxonomy that reflects the major disease and pest categories affecting arecanut palms, along with four healthy tissue classes that serve as negative controls and enable the model to distinguish between healthy and diseased states. This taxonomy was likely derived from the available dataset and represents the most commonly encountered conditions in arecanut cultivation, particularly in the Indian subcontinent where arecanut is a major commercial crop. The inclusion of healthy tissue categories for different plant organs (foot, leaf, nut, trunk) is a thoughtful design choice that allows the system to handle images captured from any part of the palm and correctly identify them as healthy, reducing false positive disease diagnoses.

ID	Class Name	Category	Scientific Name	Healthy
0	Bud Borer	Pest Infestation	Carvalhoia arecae / Manatha albipes	No
1	Healthy Foot	Healthy	Areca catechu	Yes
2	Healthy Leaf	Healthy	Areca catechu	Yes
3	Healthy Nut	Healthy	Areca catechu	Yes
4	Healthy Trunk	Healthy	Areca catechu	Yes

ID	Class Name	Category	Scientific Name	Healthy
5	Mahali Koleroga	Fungal Infection	Phytophthora meadii	No
6	Stem Cracking	Physiological	Abiotic Stress / Secondary Botryodiplodia	No
7	Stem Bleeding	Fungal Infection	Thielaviopsis paradoxa	No
8	Yellow Leaf Disease	Phytoplasma	Phytoplasma (16SrXI group)	No

Table 3: Complete nine-class taxonomy with scientific nomenclature and health status.

4.2 Disease Knowledge Base

One of the most valuable components of the YLD system is its comprehensive agronomic knowledge base embedded in the **disease_info.py** module. For each of the nine classes, the knowledge base provides the common and scientific names, a detailed textual description of the condition, a list of observable symptoms that can aid field diagnosis, recommended treatment measures including specific chemical applications and dosages, and preventive cultural practices for long-term disease management. This knowledge base transforms the system from a simple image classifier into a practical decision-support tool for agricultural extension workers and farmers.

The knowledge base covers a diverse range of pathogen types. Bud Borer is a pest infestation caused by caterpillars of Carvalhoia arecae and Manatha albipes that tunnel into the tender unopened spindle leaf. Mahali Koleroga is a devastating fungal disease caused by Phytophthora meadii that leads to massive premature nut dropping during the monsoon season. Yellow Leaf Disease, which gives the repository its name, is a systemic phytoplasma disease transmitted by plant hoppers (Proutista moesta) that causes characteristic golden yellowing and is particularly challenging because no single chemical cure exists. Stem Bleeding is caused by the fungus Thielaviopsis paradoxa and is characterized by dark reddish-brown exudation from trunk cracks. Stem Cracking is primarily an abiotic disorder caused by thermal stress but can be secondarily infected by Botryodiplodia fungi, illustrating the complex interplay between environmental and biological stressors in arecanut cultivation.

5. Grad-CAM Explainability

5.1 Explainability Mechanism

The YLD system integrates Gradient-weighted Class Activation Mapping (Grad-CAM) as its primary explainability mechanism, providing visual transparency into the model decision-making process. Grad-CAM works by computing the gradient of the target class score with respect to the feature maps of the final convolutional layer, then using these gradients as weights to produce a coarse localization map that highlights the image regions most relevant to the prediction. In the context of MobileNetV3-Small, the target layer is `features[12]`, the last convolutional block in the network architecture.

The implementation in **gradcam.py** uses the pytorch-grad-cam library to perform the gradient backpropagation and generate the activation maps. The resulting grayscale CAM mask (224 x 224) is then processed in two ways: first, it is composited onto the resized original image using the show_cam_on_image utility to produce a color-coded overlay visualization where red and yellow regions indicate high activation (areas the model found most discriminative), and blue/green regions indicate low activation. Second, the raw CAM mask is converted to a Jet-colormapped heatmap using OpenCV for standalone viewing. Both visualizations are encoded as base64 PNG data URLs and included in the API response for seamless frontend rendering.

5.2 Severity Estimation

Beyond visualization, the system implements a quantitative severity estimation mechanism that derives a numerical score from the Grad-CAM output. The severity score is calculated as the percentage of pixels in the CAM mask that exceed a configurable activation threshold (default 0.35). This represents the fraction of the image area where the model detected sufficiently strong discriminative features related to the predicted class. The score is then mapped to a four-level severity classification: Mild/Early Stage (less than 10% affected area), Moderate (10-30%), Severe (30-55%), and Critical/Advanced (greater than 55%).

This severity estimation provides an additional layer of diagnostic information beyond the simple class label. For diseased tissues, a higher severity score suggests more extensive visible symptoms, which may correlate with disease progression and inform urgency of treatment. For healthy tissues, the severity level is overridden to display "N/A - Healthy Tissue" to avoid misleading severity indicators on healthy samples. It is important to note that this severity metric is derived from model activation patterns rather than clinical pathology, and should be interpreted as an approximate indicator of disease extent rather than a precise medical measurement. Nevertheless, it provides a useful quantitative complement to the qualitative visual explanation offered by the Grad-CAM overlay.

Severity Level	Affected Area	Interpretation
Mild / Early Stage	< 10%	Initial symptoms, localized damage, high treatment success rate
Moderate	10% - 30%	Visible progression, intervention needed to prevent spread
Severe	30% - 55%	Advanced symptoms, urgent treatment required
Critical / Advanced	> 55%	Extensive damage, may require plant removal
N/A - Healthy Tissue	N/A	No disease detected, severity not applicable

Table 4: Grad-CAM severity estimation levels and their clinical interpretation.

6. API Design and Integration

6.1 Endpoint Specification

The backend exposes three REST API endpoints through FastAPI, each serving a distinct purpose in the overall system workflow. The design follows RESTful conventions with clear HTTP method semantics and well-structured JSON request and response payloads. The API includes automatic OpenAPI/Swagger documentation generation through FastAPI built-in support, making it self-documenting and easy for frontend developers to integrate with. The following table summarizes the three endpoints and their specifications:

Method	Endpoint	Description	Parameters
GET	/health	System health check, model status, class list	None
GET	/api/diseases	Full disease knowledge catalog (9 classes)	None
POST	/predict	Image classification + Grad-CAM	file (image), generate_cam (bool)

Table 5: REST API endpoint specification.

6.2 Prediction Response Structure

The prediction endpoint returns a rich JSON response object that contains far more than just the predicted class label. The response includes a boolean demo_mode flag that indicates whether the system is operating with a trained model or in fallback mode, the predicted class identifier and human-readable name, a confidence score between 0 and 1, a boolean is_healthy flag for quick filtering, a full probability distribution dictionary mapping all nine class names to their respective probabilities, the complete disease information object from the knowledge base for the predicted class, and an optional Grad-CAM object containing base64-encoded overlay and heatmap images along with severity metrics.

This comprehensive response structure enables the frontend to present rich, multi-faceted diagnostic information without requiring additional API calls. The inclusion of the full probability distribution is particularly valuable for transparency, as it allows users to see the confidence margin between the top prediction and alternative classes, which can help identify uncertain predictions that may warrant manual expert review. The demo mode fallback mechanism ensures that the system remains functional even when the model checkpoint is not available, returning a uniform probability distribution and a clear warning message rather than crashing or returning an error.

7. Code Quality and Technical Assessment

7.1 Strengths

The YLD/AdikeScan project demonstrates several notable strengths that reflect good software engineering and machine learning practices. The separation of concerns between the backend modules is well-executed, with each file having a clear and focused responsibility. The disease knowledge base is exceptionally detailed and represents genuine domain expertise in arecanut pathology, providing practical value beyond the machine learning component. The Grad-CAM integration goes beyond a basic implementation by adding quantitative

severity estimation, making the explainability output more actionable for agricultural practitioners.

The training pipeline follows best practices for transfer learning, including appropriate data augmentation, learning rate scheduling, label smoothing, and best-model checkpointing. The rapid convergence to 99.52% validation accuracy with minimal overfitting (as evidenced by the small train-val accuracy gap) suggests that the hyperparameter choices were well-tuned. The frontend design is polished and user-friendly, with a dark emerald theme that creates a professional appearance, drag-and-drop and camera capture support that enhances usability in field conditions, and a well-organized tabbed results interface that presents complex diagnostic information in an accessible format.

7.2 Areas for Improvement

Despite its strengths, the project has several areas that could benefit from further development. The hardcoded absolute file paths in main.py and train.py (pointing to /home/gopalkrishnajs/Projects/Integer/) prevent the code from being portable and would break for any other developer cloning the repository. These should be replaced with relative paths or environment variable-based configuration. The project lacks any form of automated testing, including unit tests for the API endpoints, model inference validation, and frontend component tests, which is a significant gap for a production-oriented application.

The evaluation methodology is limited to overall accuracy and does not include per-class metrics (precision, recall, F1-score) or a confusion matrix, which are essential for understanding model performance on imbalanced datasets. There is no indication of how the dataset was collected, annotated, or validated by domain experts, which raises questions about label quality and potential annotation biases. Additionally, the system does not appear to have any form of input validation beyond checking the file content type, leaving it potentially vulnerable to malformed or adversarial inputs. The absence of a requirements.txt or environment configuration file makes dependency management and reproducibility difficult.

7.3 Technology Stack Summary

Component	Technology	Version	Purpose
Backend Framework	FastAPI	-	REST API server
Deep Learning	PyTorch	-	Model training and inference
Model Architecture	MobileNetV3-Small	-	Image classification backbone
Explainability	pytorch-grad-cam	-	Grad-CAM heatmap generation
Image Processing	OpenCV + Pillow	-	Image transforms and colormap
Frontend Framework	React	19.0.0	User interface
Build Tool	Vite	6.2.0	Dev server and bundler
Icons	lucide-react	1.16.0	UI icon library

Table 6: Complete technology stack with versions and purposes.

8. Recommendations

8.1 Short-Term Improvements

Several immediate improvements could significantly enhance the project quality and developer experience. First, replacing all hardcoded absolute paths with configurable relative paths or environment variables would make the code portable and allow other developers to run the project without modification. Second, adding a requirements.txt file (or pyproject.toml) listing all Python dependencies with version constraints would ensure reproducible environments. Third, implementing comprehensive evaluation metrics including a confusion matrix, per-class precision, recall, and F1-score would provide a much clearer picture of model performance, particularly for minority disease classes that may be underrepresented in the dataset.

Fourth, adding a basic test suite using pytest for the backend API endpoints and model loading/inference functions would catch regressions and increase confidence in code correctness. Fifth, implementing Docker Compose configuration for both the backend and frontend would simplify deployment and ensure consistent runtime environments across development and production. Sixth, adding input validation beyond content type checking, such as image dimension validation and file size limits, would improve system robustness against malformed or malicious inputs.

8.2 Long-Term Development

For longer-term development, several architectural enhancements could elevate the system from a research prototype to a production-ready agricultural tool. Implementing user authentication and a prediction history database would enable tracking of disease outbreaks over time and across geographic locations, providing valuable epidemiological data. Adding support for batch image processing would allow farmers or extension workers to classify multiple images in a single session, improving workflow efficiency. Exploring model quantization (INT8) and ONNX export would enable deployment on mobile devices and edge hardware, bringing the diagnostic capability directly to the field without requiring internet connectivity.

Expanding the disease taxonomy to include additional arecanut conditions such as Foot Rot (Ganoderma), Yellow Leaf Spot, and Nut Fall would increase the system diagnostic coverage and practical utility. Integrating with weather data APIs to provide contextual risk assessments based on current environmental conditions (humidity, temperature, rainfall) could enable proactive disease forecasting rather than reactive diagnosis. Finally, developing a multilingual interface supporting Kannada, Malayalam, and other regional languages would make the system accessible to the farming communities who need it most, as many arecanut farmers in India may not be comfortable with English-only interfaces.